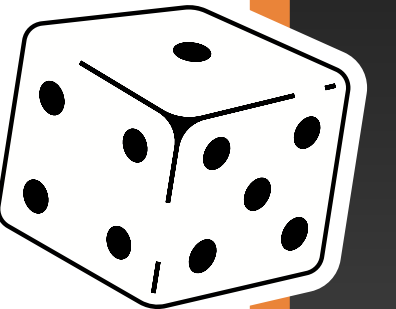


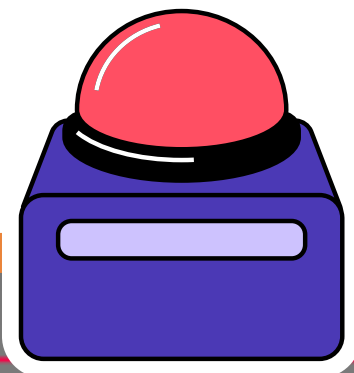


INTERFACE, ABSTRACTION, INNER CLASSES



ALGORITHMS & OBJECT-ORIENTED PROGRAMMING
WEEK 11 LAB
JOSEPH ANDREAS

FOR INTERNAL USE ONLY



AGENDA

1. INTERFACE

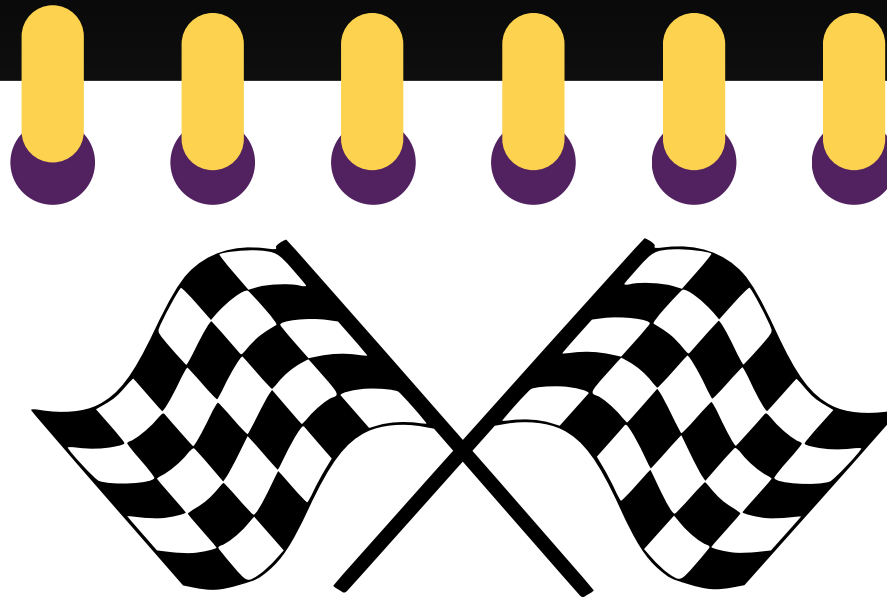
2. ABSTRACTION

3. INNER CLASSES

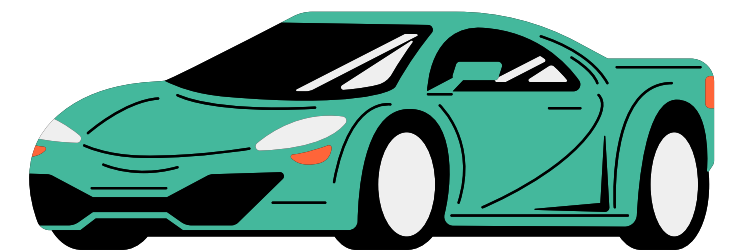
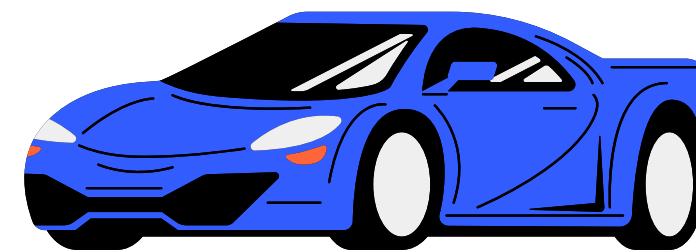
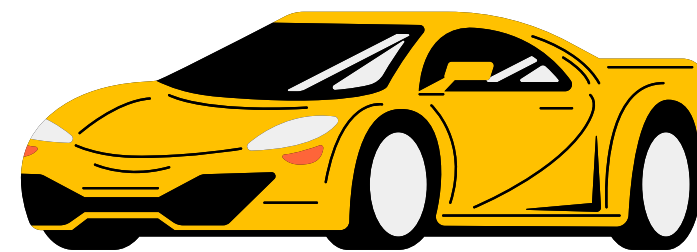
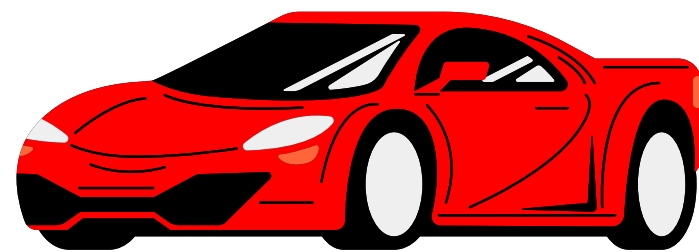


START





INTERFACE



WHAT IS AN INTERFACE ?

- Interface:
 - Blueprint of a class. Contains abstract methods (methods without a body) and constants
 - Defines what a class must do (behavior), but not how (does not provide implementation)
- A class must implement the interface and provide the actual code for the methods
- A class can implement multiple interfaces → solves the problem of multiple inheritance

EXAMPLE OF INTERFACE

```
interface Payment {  
    void pay(double amount);  
}  
  
class CreditCard implements Payment  
{  
    public void pay(double amount) {  
        System.out.println("Paid $" +  
amount + " using Credit Card");  
    }  
}  
  
class PayPal implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid $" +  
amount + " using PayPal");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Payment p1 = new PayPal();  
        Payment p2 = new CreditCard();  
        p1.pay(99.99);  
        p2.pay(49.99);  
    }  
}
```

Payment is the interface → Defines what methods must exist

CreditCard and **Paypal** both implement the **Payment** interface

Payment p1 = new **PayPal**();

- **Payment** is the reference type (what we expect)
- new **PayPal**() is the object type (the actual behavior)

Interface type is used so all payment methods can be treated the same way

IMPLEMENTING MULTIPLE INTERFACES

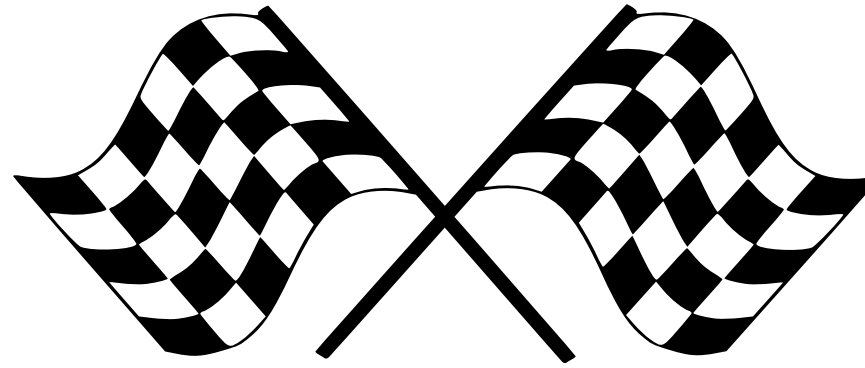
```
interface Loginable {  
    void login();  
}  
  
interface Purchasable {  
    void purchase();  
}  
  
class OnlineUser implements Loginable, Purchasable {  
    public void login() {  
        System.out.println("User logged in.");  
    }  
  
    public void purchase() {  
        System.out.println("User made a purchase.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        OnlineUser user = new OnlineUser();  
        user.login();  
        user.purchase();  
    }  
}
```

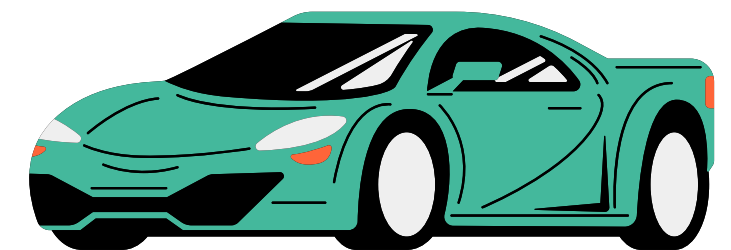
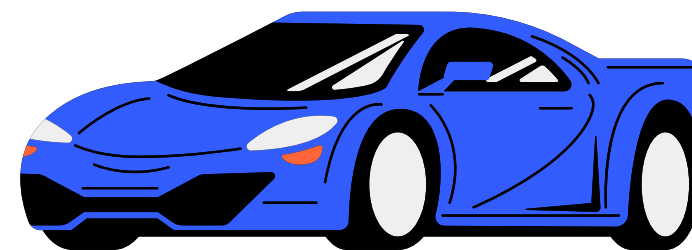
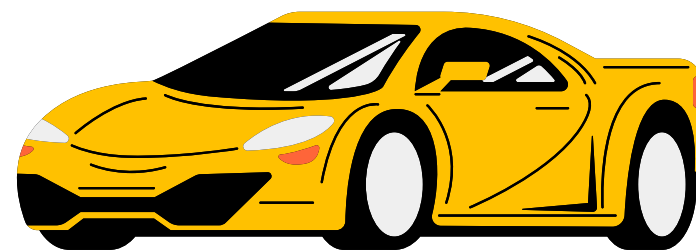
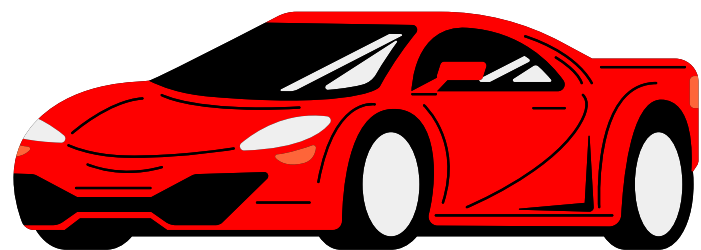
`OnlineUser` implements both `Loginable` and `Purchasable`, and overrides the provided methods in the same class

WHEN TO USE INTERFACE VS INHERITANCE

- Use interface when:
 - Need multiple inheritance (extending multiple classes are not allowed)
 - Classes are unrelated but must share behavior (Bird, Airplane, Superhero all implement Flyable)
 - Define capabilities, not objects (Flyable, Drivable, Runnable, etc)
- Use inheritance when:
 - A real “is-a” relationship exists (Dog is an Animal, Car is a Vehicle)
 - Want polymorphism with shared functionality (parent class contains code → children inherit it)
 - Subclasses differ only in specific parts (parent class: common code, child class: unique parts)



ABSTRACTION



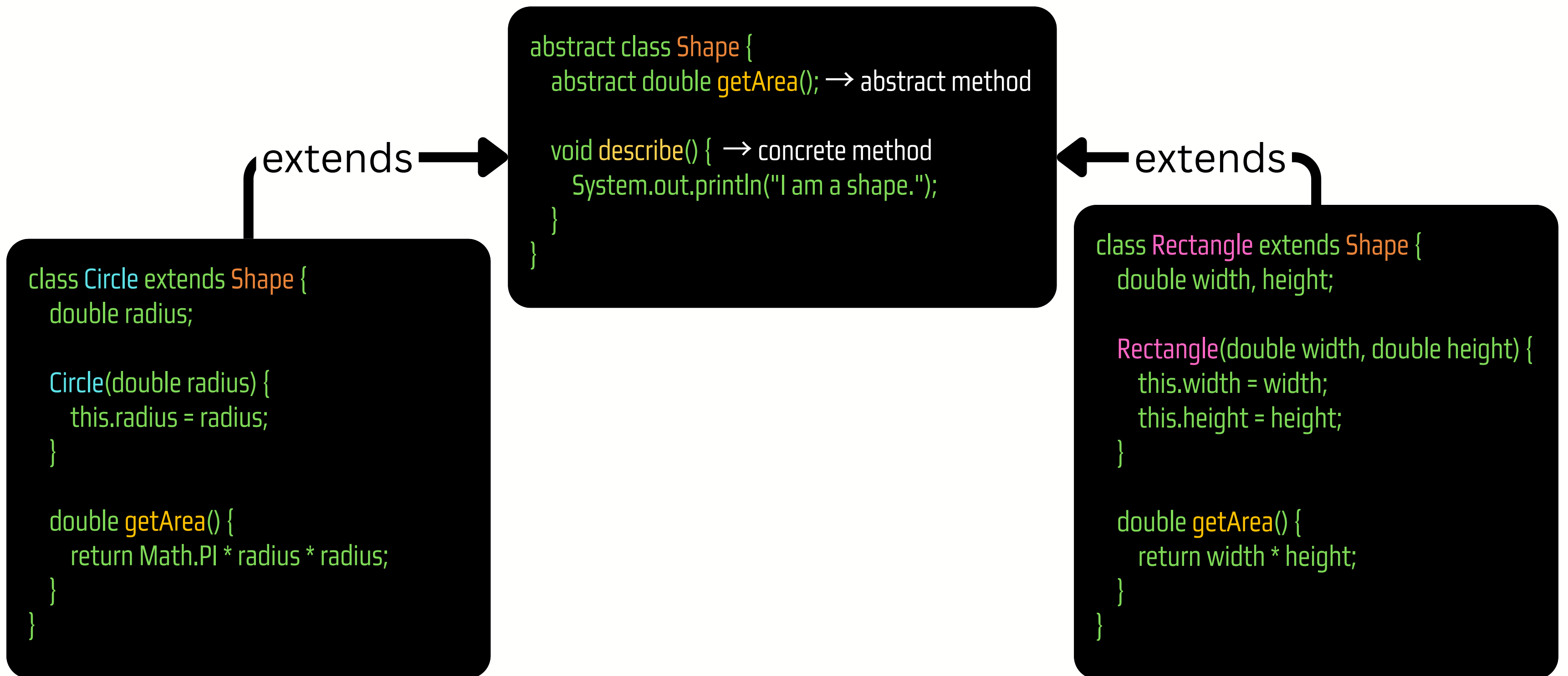
WHAT IS ABSTRACTION ?

Abstraction = Hiding complex details and showing only what's necessary

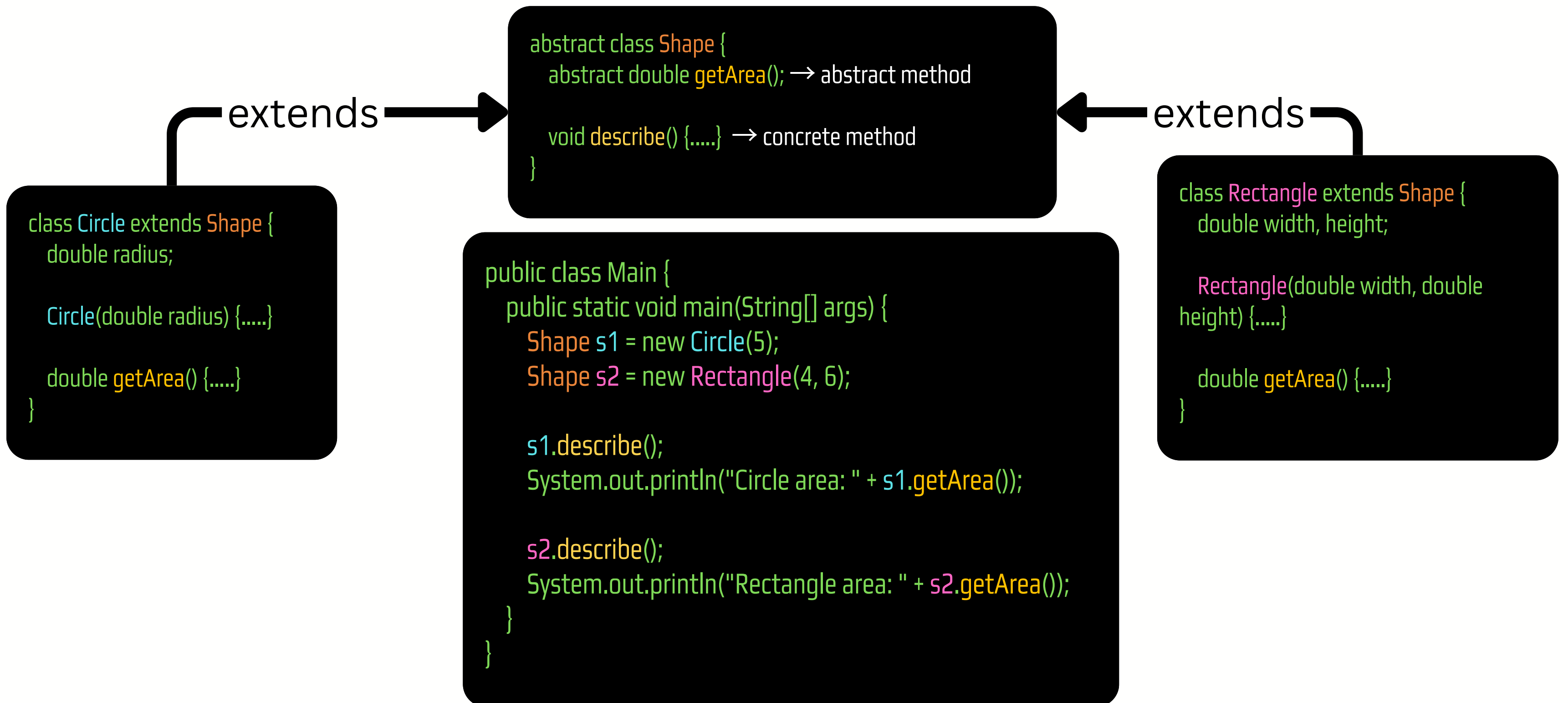
In Java there are two ways to do abstraction:

- Abstract classes, characteristics:
 - Cannot be instantiated
 - Can have abstract and non-abstract methods
 - Can have variables, constructors, and concrete method implementations
 - Partially abstract (mix of abstract + normal methods)
- Interfaces: 100% abstract

EXAMPLE OF ABSTRACTION



EXAMPLE OF ABSTRACTION



WHEN TO USE AN ABSTRACT CLASS

Use abstract class when:

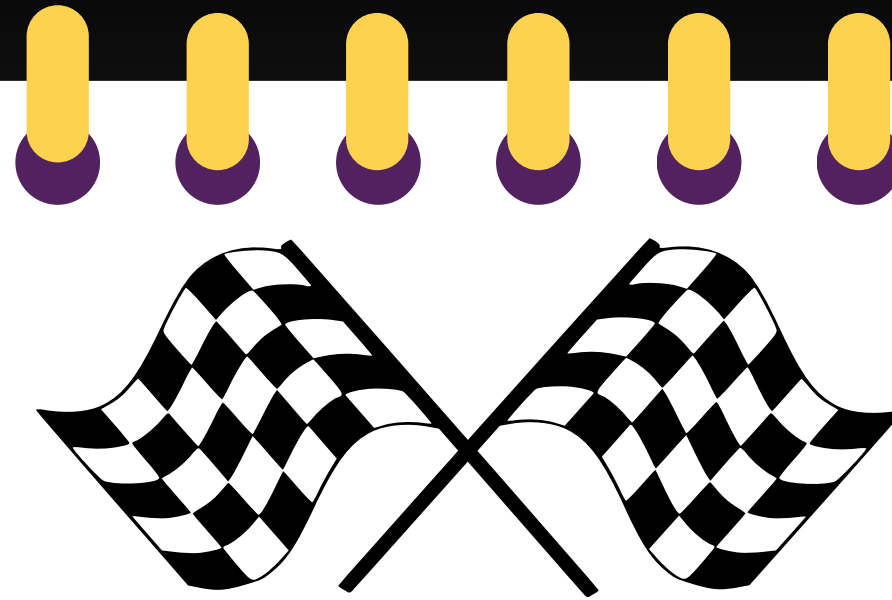
- Want to provide shared code that subclasses can use
- Define a base class with common fields/constructors
- Want to mix abstract + concrete methods
- Want strong hierarchical relationships (e.g.: Dog is an Animal)

Use interfaces when:

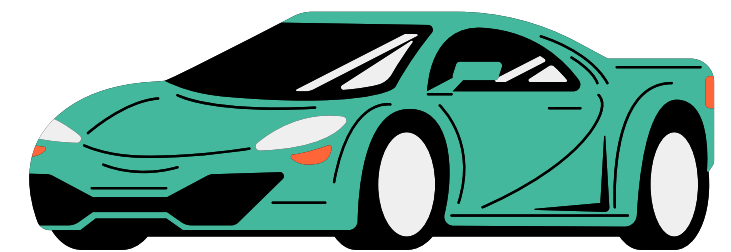
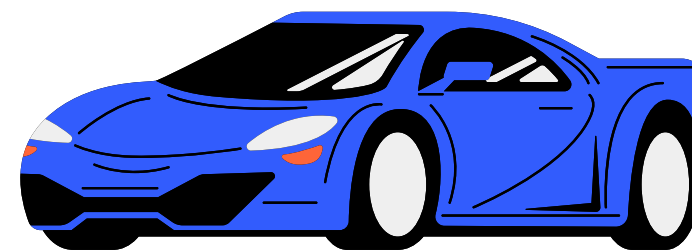
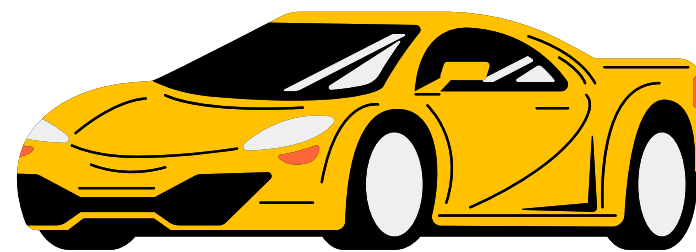
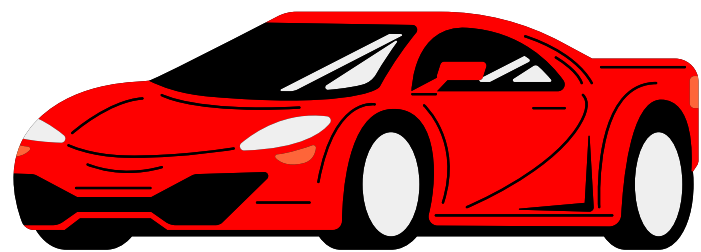
- Want multiple inheritance of type
- Don't need shared fields or constructors
- Want to define a capability
- Want unrelated classes to share the same behavior (e.g.: Driveable, Playable)

Very short summary:

- Interface = What to do
- Abstract Class = What to do + Some of how to do it



INNER CLASSES



CLASS INSIDE A CLASS => INNER CLASSES

Inner classes are classes inside another class, used to group logic together or create one-time implementations

Why use inner classes?

- Group classes that belong together (grouping related logic)
- Reducing unnecessary classes in the project
- Organize code more clearly

EXAMPLE OF INNER CLASS

```
class Car {  
    String model = "Honda";  
  
    class Engine {  
        void start() {  
            System.out.println(model + " engine started.");  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car car = new Car();  
        Car.Engine engine = car.new Engine();  
        engine.start();  
    }  
}
```

Car is a class → Contains data and inner classes

Engine is an inner class inside **Car** → Can access **Car's** fields: Inner class automatically gets access to model

Car car = new **Car**();

Creates an object of the outer class

Car.Engine engine = car.new **Engine**();

To create an inner-class object, you must use an instance of the outer class

engine.start() → Prints model + message

Uses model from outer **Car** class

LOCAL INNER CLASS

```
class Greeting {  
    void sayGreeting() {  
  
        class LocalGreeter {  
            void greet() {  
                System.out.println("Hello from  
a local inner class!");  
            }  
        }  
  
        LocalGreeter greeter = new  
LocalGreeter();  
        greeter.greet();  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Greeting g = new Greeting();  
        g.sayGreeting();  
    }  
}
```

In main():

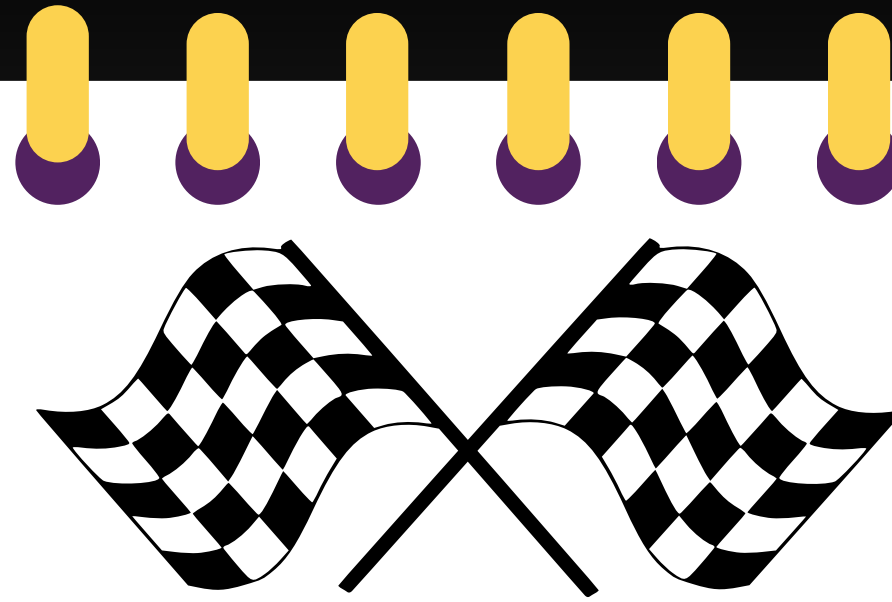
- `Greeting g = new Greeting();`
→ Create `Greeting` object
- `g.sayGreeting();` → Runs
method which defines and
uses the local inner class

`Greeting` is a normal class → Contains a
method `sayGreeting()`

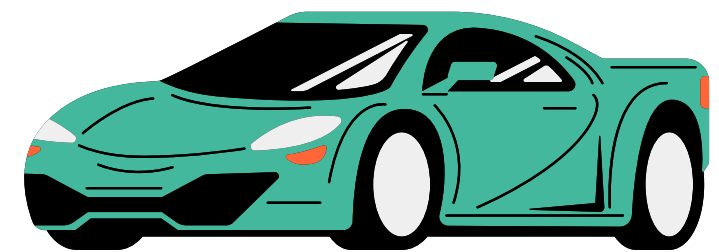
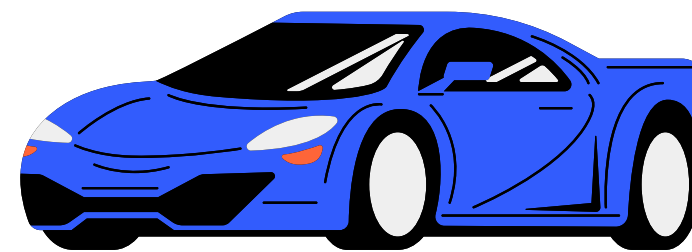
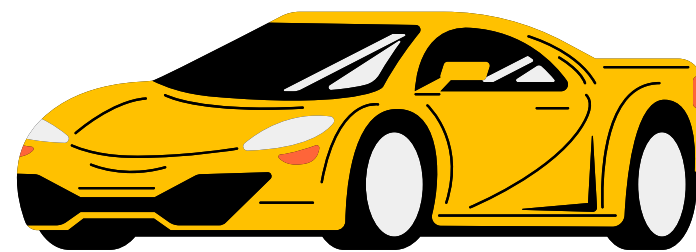
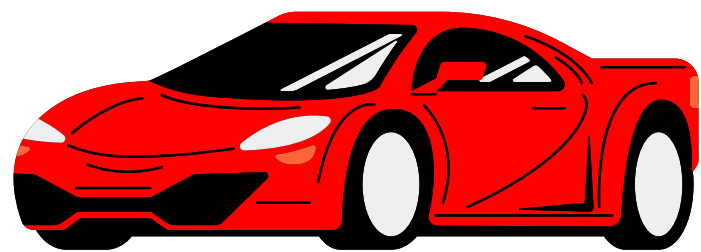
`LocalGreeter` is a local inner class → Defined
inside a method: Exists only inside
`sayGreeting()` and cannot be used outside it

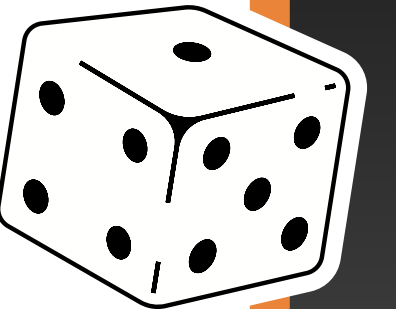
`LocalGreeter` can access method scope and
enclosing class members → It can use
anything inside `sayGreeting()`

`LocalGreeter greeter = new LocalGreeter();`
Object of the local inner class is created inside
the method only.



ANY QUESTIONS?





THANK YOU FOR
YOUR ATTENTION!

